

Documentazione del Progetto di Basi di Dati

Lafragola Daniele - N86005559

24 marzo 2026

Indice

1	Introduzione	2
1.1	Descrizione del Progetto	2
2	Modello Concettuale	3
2.1	Ristrutturazione	3
2.1.1	Eliminazione delle ridondanze	4
2.1.2	Eliminazione delle generalizzazioni/specializzazioni	4
2.1.3	Analisi degli attributi multivalore	4
2.1.4	Analisi degli attributi strutturati	4
2.1.5	Analisi delle chiavi primarie	5
2.2	Class Diagram Ristrutturato	6
3	Dizionari	8
3.1	Dizionario delle Classi	8
3.2	Dizionario delle Associazioni	10
3.3	Dizionario dei Vincoli	12
4	Modello Logico	13
4.1	Schema Logico	13
5	Progettazione Fisica	13
5.1	Definizioni delle Tabelle	13
5.1.1	Tabella Utente	14
5.1.2	Tabella Passeggero	14
5.1.3	Tabella Prenotazione	14
5.1.4	Tabella Volo Nazionale	15
5.1.5	Tabella Volo Internazionale	16
5.1.6	Tabella Gate	17

5.2	Funzioni dei Trigger	17
5.2.1	Tabella Prenotazione	17
5.2.2	Tabella Volo Nazionale/Volo Internazionale	21

1 Introduzione

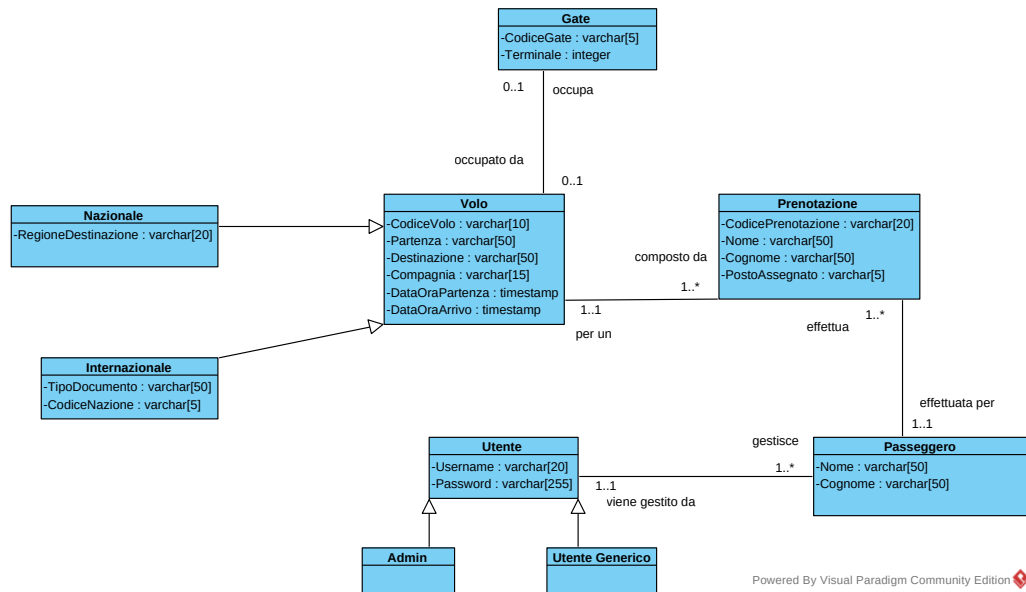
Questo elaborato documenta la progettazione e la realizzazione di una base di dati relazionale del DBMS PostgreSQL. Ad opera dello studente Lafragola Daniele, del Corso di Laurea in Informatica presso l'Università degli Studi di Napoli Federico II. Il database ha lo scopo di rappresentare e gestire un sistema informativo aeroportuale.

1.1 Descrizione del Progetto

Esso permette l'accesso a diverse tipologie di utenti, come utenti facenti parte dello staff amministrativo, che possono visualizzare e gestire informazioni sui voli, le prenotazioni e i gate, e utenti comuni/passeggeri, che possono visualizzare e prenotare un volo o più voli. Il sistema inoltre, divide i voli tra voli nazionali e internazionali. Il sistema tiene traccia dei passeggeri registrati, delle loro prenotazioni effettuate, dei voli previsitati e dei gate associati alla partenza dei voli.

2 Modello Concettuale

Il modello concettuale è stato realizzato utilizzando il diagramma E-R (Entità-Relazione).



2.1 Ristrutturazione

Il precedente Class Diagram andrà ora ristrutturato per permettere la sua implementazione. La ristrutturazione seguirà questi passaggi:

- Eliminazione delle ridondanze
- Eliminazione delle generalizzazioni/specializzazioni
- Analisi degli attributi multivalore
- Analisi degli attributi strutturati
- Analisi delle chiavi primarie

2.1.1 Eliminazione delle ridondanze

Le ridondanze rappresentano degli attributi che possono essere facilmente ricavabili o calcolati da altri presenti. Nello schema, le entità Prenotazione e Passeggero condividono attributi del passeggero, come Nome e Cognome. Nella entità Prenotazione, dunque, queste informazioni sono ridondanti perché già presenti nell'entità Passeggero. La soluzione più corretta, è quella di eliminare gli attributi ridondanti dall'entità Prenotazione.

2.1.2 Eliminazione delle generalizzazioni/specializzazioni

Le generalizzazioni rappresentano delle entità (Figlie) che ereditano attributi e relazioni da un'altra entità (Padre). Nel nostro caso, sono presenti due diverse generalizzazioni/specializzazioni:

- La specializzazione dell'entità Utente, in Admin e Utente Generico.
- La specializzazione dell'entità Volo, in Volo Nazionale e Volo Internazionale.

Per l'eliminazione di queste generalizzazioni, si è deciso di adottare due strategie differenti:

1. Per la specializzazione dell'entità Utente, la scelta migliore è quella dell'accorpamento. In questo modo, si manterranno tutte le informazioni in un'unica tabella, aggiungendo un attributo di tipo booleano per distinguere tra Admin e Utente Generico.
2. Per la specializzazione dell'entità Volo, si è deciso di adottare la strategia del partizionamento. In questo modo, saranno presenti due entità distinte per i voli nazionali e quelli internazionali, riducendo anche il possibile numero di operazioni richieste.

2.1.3 Analisi degli attributi multivalore

Gli attributi multivalore sono quegli attributi che possono assumere più di un valore per una singola istanza dell'entità. Nello schema, non sono presenti attributi multivalore, per tanto non è necessario effettuare alcuna operazione.

2.1.4 Analisi degli attributi strutturati

Gli attributi strutturati sono quegli attributi che posseggono più componenti. Anche in questo caso, non sono presenti attributi strutturati nello schema.

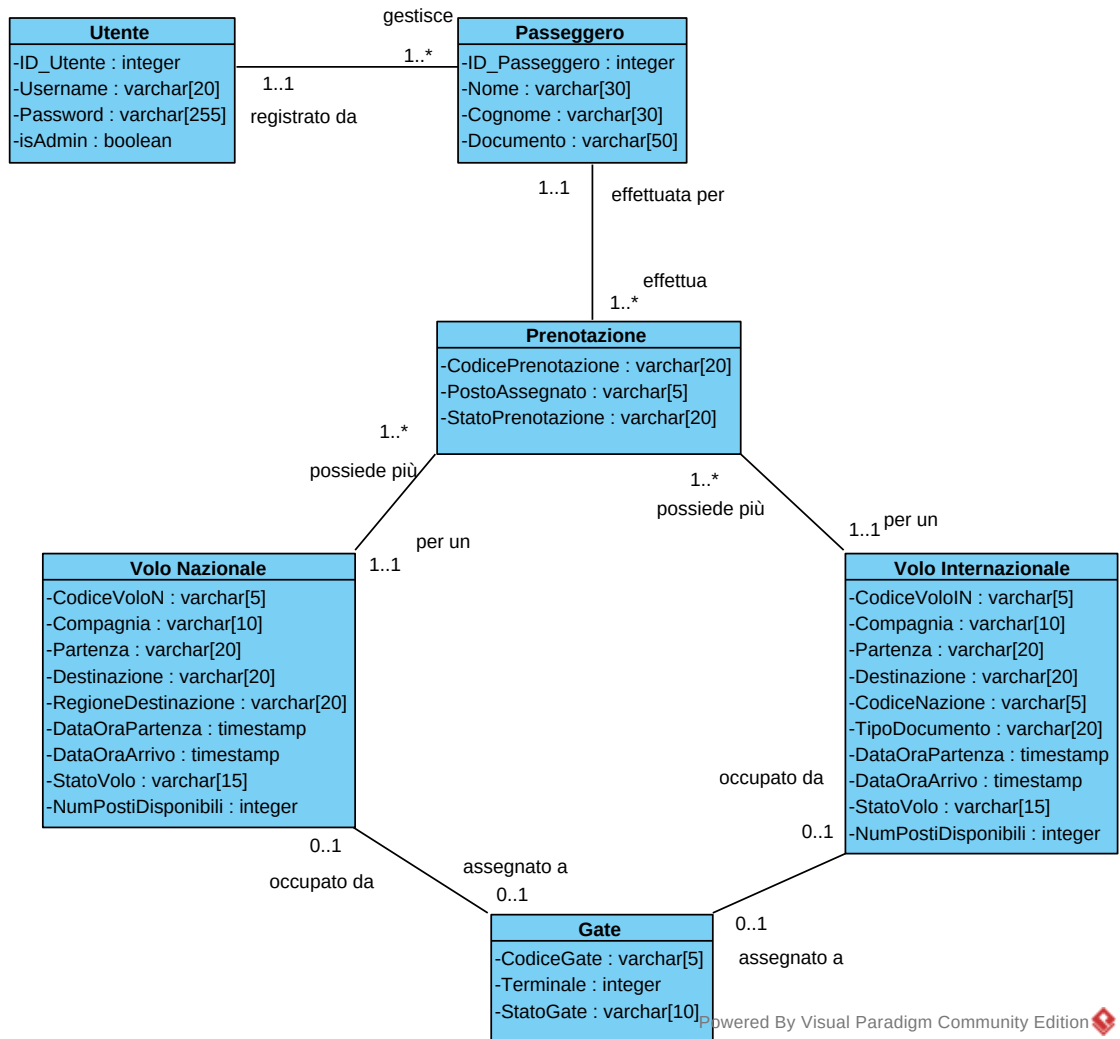
2.1.5 Analisi delle chiavi primarie

Nello schema, sono presenti entità ai quali è consigliabile aggiungere una chiave primaria "surrogata", ovvero un attributo composto da un intero autoincrementale, che identifica quella specifica istanza dell'entità. Queste entità sono:

- Utente → Aggiunta dell'attributo "ID_Utente".
- Passeggero → Aggiunta dell'attributo "ID_Passeggero".

Per le entità Prenotazione e Gate, si è deciso di usare l'attributo "CodicePrenotazione" e "CodiceGate" come chiavi primaria, in quanto identificano univocamente ogni prenotazione e ogni gate. Per le entità Volo Nazionale e Volo Internazionale, si è deciso di usare l'attributo "CodiceVolo" come chiave primaria, in quanto esso identifica univocamente ogni volo. Avendo però due entità distinte, la chiave "CodiceVolo" è stata rinominata in "CodiceVoloN" per i voli nazionali e "CodiceVoloIN" per i voli internazionali.

2.2 Class Diagram Ristrutturato



3 Dizionari

3.1 Dizionario delle Classi

Classe	Descrizione	Attributi e Vincoli
Utente	Rappresenta un utente del sistema, sia generico che amministratore.	• ID_Utente (integer, PK): Chiave primaria surrogata che identifica univocamente un utente. • Username (varchar[20]): Nome utente. • Password (varchar[255]): Password dell'utente. • isAdmin (boolean): Flag per i permessi amministrativi.
Passeggero	Rappresenta un passeggero con i suoi dati personali.	• ID_Passeggero (integer, PK): Chiave primaria surrogata identificativa. • Nome (varchar[30]): Nome del passeggero. • Cognome (varchar[30]): Cognome del passeggero. • Documento (varchar[50]): Tipologia di documento d'identità.
Prenotazione	Rappresenta una prenotazione effettuata per un volo.	• CodicePrenotazione (varchar[20], PK): Chiave primaria, codice univoco dell'operazione. • PostoAssegnato (varchar[10]): Identificativo del posto a sedere. • StatoPrenotazione (varchar[20]): Stato (Confermata, Cancellata, in Attesa).

Classe	Descrizione	Attributi e Vincoli
Volo	Rappresenta un volo su territorio nazionale.	• CodiceVoloN (varchar[5], PK): Identificativo univoco nazionale. • Compagnia (varchar[10]): Compagnia aerea. • Partenza / Destinazione (varchar[20]): Città di origine e arrivo. • RegioneDestinazione (varchar[20]): Regione italiana di arrivo. • DataOraPartenza / Arrivo (timestamp): Orari previsti. • StatoVolo (varchar[15]): in Attesa, Programmato, Cancellato, Decollato, In Ritardo. • NumPostiDisponibili (integer): Posti ancora acquistabili.
Volo	Rappresenta un volo su territorio internazionale.	• CodiceVoloIN (varchar[5], PK): Identificativo univoco internazionale. • Compagnia (varchar[10]): Compagnia aerea. • Partenza / Destinazione (varchar[20]): Città di origine e arrivo. • CodiceNazione (varchar[5]): Codice dello stato di destinazione. • TipoDocumento (varchar[20]): Passaporto, Visto o Carta d'Identità. • DataOraPartenza / Arrivo (timestamp): Orari previsti. • StatoVolo (varchar[15]): Stato operativo del volo. • NumPostiDisponibili (integer): Posti ancora acquistabili.
Gate	Rappresenta un gate dell'aeroporto, con le sue informazioni di localizzazione e stato.	• CodiceGate (varchar[5]): Codice nominativo del gate. • Terminale (integer): Identifica il terminale di appartenenza del gate. • StatoGate (varchar[10]): Stato operativo (Aperto, Chiuso o Occupato).

3.2 Dizionario delle Associazioni

Gestione Profili (Utente-Passeggero): Associazione 1 : N . Un *Utente* gestisce molteplici *Passeggero* (1..*). Ogni *Passeggero* appartiene a un solo *Utente* (1..1).

Titolarità (Passeggero-Prenotazione): Associazione 1 : N . Un *Passeggero* è titolare di diverse *Prenotazioni* (1..*). Ogni *Prenotazione* è legata a un singolo *Passeggero* (1..1).

Assegnazione Volo (Prenotazione-Volo): Associazione 1 : N . Ogni *Prenotazione* garantisce un posto su un unico *Volo* (1..1). Ciascun *Volo* riceve prenotazioni da numerosi passeggeri (1..*).

Assegnazione Gate (Volo-Gate): Associazione 1 : 1. Un *Volo* è associato a un *Gate* (0..1). Un *Gate* è dedicato a un solo volo alla volta (0..1).

3.3 Dizionario dei Vincoli

Tipo di Vincolo	Descrizione	Attributi Coinvolti
Integrità Primaria (PK)	Garantisce l'univocità delle righe, identificando ogni record in modo certo e non nullo.	• ID_Utente, ID_Passeggero, CodicePrenotazione, CodiceGate: ID univoci per le entità. • CodiceVoloN, CodiceVoloIN: Identificativi univoci per i voli.
Integrità Referenziale (FK)	Assicura la consistenza dei legami tra le tabelle, forzando la corrispondenza tra chiavi.	• ID_Utente: Lega Passeggero e Prenotazione all'account. • ID_Passeggero: Associa la Prenotazione al viaggiatore. • CodiceVoloN/IN: Collegano la Prenotazione alla tratta. • CodiceGate: Associa i Voli al punto di imbarco.
Unicità (UNIQUE)	Garantisce che i valori in una colonna siano unici in tutta la tabella.	• Username: Impedisce account duplicati. • Volo/Posto: Impedisce l'assegnazione doppia dello stesso posto sul medesimo volo.
Controllo (CHECK)	Definisce condizioni logiche personalizzate per la validità dei dati.	• Cronologia Orari: Verifica che l'arrivo sia successivo alla partenza. • Esclusività Tratta: Una prenotazione è o Nazionale o Internazionale. • Stato Entità: Limita i valori per Stati di Gate, Prenotazione e Volo.
Integrità (NOT NULL)	Impone l'obbligatorietà di inserimento per i campi essenziali.	• Dati Anagrafici: Nome, Cognome e Documento. • Autenticazione: Password dell'utente. • Logistica: Città di partenza e destinazione.

4 Modello Logico

Si arriva quindi alla fase di modellazione logica, in cui il modello concettuale viene tradotto in uno schema relazionale.

4.1 Schema Logico

Il seguente schema relazionale rappresenta la struttura logica della base di dati. Nello specifico, le chiavi primarie sono indicate con una singola sottolineatura, mentre le chiavi esterne sono indicate con una doppia sottolineatura.

- **UTENTE** (ID_Utente, Username, Password, isAdmin)
- **PASSEGGERO** (ID_Passeggero, Nome, Cognome, Documento, ID_Utente)
- **GATE** (CodiceGate, Terminale, StatoGate)
- **VOLO_NAZIONALE** (CodiceVoloN, Compagnia, Partenza, Destinazione, RegioneDestinazione, DataOraPartenza, DataOraArrivo, StatoVolo, NumPostiDisponibili, CodiceGate)
- **VOLO_INTERNAZIONALE** (CodiceVoloIN, Compagnia, Partenza, Destinazione, CodiceNazione, TipoDocumento, DataOraPartenza, DataOraArrivo, StatoVolo, NumPostiDisponibili, CodiceGate)
- **PRENOTAZIONE** (CodicePrenotazione, PostoAssegnato, StatoPrenotazione, ID_Passeggero, ID_Utente, CodiceVoloN, CodiceVoloIN)

5 Progettazione Fisica

La progettazione fisica consiste nella traduzione dello schema logico in un insieme di comandi SQL per la creazione delle tabelle e dei vincoli nel DBMS PostgreSQL.

5.1 Definizioni delle Tabelle

Di seguito è riportato il codice SQL per la creazione delle tabelle del database.

5.1.1 Tabella Utente

```
1 CREATE TABLE utente (  
2   id_utente serial,  
3   username varchar(20) NOT NULL,  
4   password varchar(255) NOT NULL,  
5   isadmin bool NOT NULL,  
6   CONSTRAINT utente_pkey PRIMARY KEY (id_utente),  
7   CONSTRAINT utente_username_key UNIQUE (username)  
8 );
```

5.1.2 Tabella Passeggero

```
1 CREATE TABLE passeggero (  
2   id_passeggero serial,  
3   nome varchar(30) NOT NULL,  
4   cognome varchar(30) NOT NULL,  
5   documento varchar(50) NOT NULL,  
6   id_utente integer,  
7   CONSTRAINT passeggero_pkey PRIMARY KEY (id_passeggero),  
8   CONSTRAINT passeggero_id_utente_fkey FOREIGN KEY (  
   id_utente) REFERENCES utente(id_utente) ON DELETE  
   CASCADE  
9 );
```

5.1.3 Tabella Prenotazione

```
1 CREATE TABLE prenotazione (  
2   codiceprenotazione varchar(20) NOT NULL,  
3   postoassegnato varchar(5) NOT NULL,  
4   statoprenotazione varchar(20) DEFAULT 'inAttesa',  
5   id_passeggero integer NOT NULL,  
6   id_utente integer NOT NULL,  
7   codicevolon varchar(5),  
8   codicevoloin varchar(5),  
9   CONSTRAINT prenotazione_pkey PRIMARY KEY (  
   codiceprenotazione),  
10  CONSTRAINT chk_stato_prenotazione CHECK (  
   StatoPrenotazione IN ('Confermata', 'Cancellata', '  
   inAttesa'))),  
11  CONSTRAINT prenotazione_codicevoloin_fkey FOREIGN KEY (  
   codicevoloin) REFERENCES volo_internazionale(  
   codicevoloin),
```

```
12  CONSTRAINT prenotazione_codicevolon_fkey FOREIGN KEY (
13      codicevolon) REFERENCES volo_nazionale(codicevolon),
14  CONSTRAINT prenotazione_id_passeggero_fkey FOREIGN KEY
15      (id_passeggero) REFERENCES passeggero(id_passeggero),
16  CONSTRAINT prenotazione_id_utente_fkey FOREIGN KEY (
17      id_utente) REFERENCES utente(id_utente)
18 );
19
20 --TRIGGER SULLA TABELLA
21 CREATE TRIGGER trg_controllo_documenti
22 BEFORE INSERT OR UPDATE ON prenotazione
23 FOR EACH ROW
24 EXECUTE FUNCTION controllo_documenti_internazionali();
25
26 CREATE TRIGGER trg_prenotazione_insert
27 BEFORE INSERT ON prenotazione
28 FOR EACH ROW
29 EXECUTE FUNCTION gestione_prenotazione_completa();
30
31 CREATE TRIGGER trg_ripristino_posti
32 AFTER DELETE OR UPDATE ON prenotazione
33 FOR EACH ROW
34 EXECUTE FUNCTION gestione_ripristino_posti_unificata();
```

5.1.4 Tabella Volo Nazionale

```
1  CREATE TABLE volo_nazionale (
2      codicevolon varchar(5) NOT NULL,
3      compagnia varchar(10) NOT NULL,
4      partenza varchar(20) NOT NULL,
5      destinazione varchar(20) NOT NULL,
6      regionedestinazione varchar(20) NOT NULL,
7      dataorapartenza timestamp NOT NULL,
8      dataoraarrivo timestamp NOT NULL,
9      statovolo varchar(20),
10     codicegate varchar(5),
11     numpostidisponibili integer,
12     CONSTRAINT chk_scalo_napoli CHECK (Partenza = 'Napoli'
13         OR Destinazione = 'Napoli'),
14     CONSTRAINT volo_nazionale_check1 CHECK ((dataoraarrivo
15         > dataorapartenza)),
16     CONSTRAINT volo_nazionale_numpostidisponibili_check
17         CHECK ((numpostidisponibili <= 150)),
```

```

15  CONSTRAINT volo_nazionale_pkey PRIMARY KEY (codicevolon
    ),
16  CONSTRAINT chk_stato_volo CHECK (StatoVolo IN ('
    Programmato', 'in Attesa', 'Decollato', 'Cancellato',
    'in Arrivo', 'in Ritardo'))
17  CONSTRAINT volo_nazionale_codicegate_fkey FOREIGN KEY
    (codicegate) REFERENCES gate(codicegate) ON DELETE
    CASCADE
18 );
19
20 --TRIGGER SULLA TABELLA
21 CREATE TRIGGER trg_assegna_gate_n
22 BEFORE UPDATE OF StatoVolo ON volo_nazionale
23 FOR EACH ROW
24 EXECUTE FUNCTION assegna_gate_automatico();
25
26 CREATE TRIGGER trg_libera_gate_n
27 AFTER UPDATE OF StatoVolo OR DELETE ON volo_nazionale
28 FOR EACH ROW
29 EXECUTE FUNCTION libera_gate_automatico();

```

5.1.5 Tabella Volo Internazionale

```

1  CREATE TABLE volo_internazionale (
2    codicevoloin varchar(5) NOT NULL,
3    compagnia varchar(10) NOT NULL,
4    partenza varchar(20) NOT NULL,
5    destinazione varchar(20) NOT NULL,
6    codicenazione varchar(5) NOT NULL,
7    tipodocumento varchar(20) NULL,
8    dataorapartenza timestamp NOT NULL,
9    dataoraarrivo timestamp NOT NULL,
10   statovolo varchar(20),
11   codicegate varchar(5),
12   numpostidisponibili integer,
13   CONSTRAINT chk_scalo_napoli_int CHECK (Partenza = '
    Napoli' OR Destinazione = 'Napoli'),
14   CONSTRAINT volo_internazionale_check1 CHECK ((
    dataoraarrivo > dataorapartenza)),
15   CONSTRAINT volo_internazionale_pkey PRIMARY KEY (
    codicevoloin),

```



```

16  CONSTRAINT chk_stato_volo CHECK (StatoVolo IN ('
    Programmato', 'in Attesa', 'Decollato', 'Cancellato',
    'in Arrivo', 'in Ritardo'))
17  CONSTRAINT chk_documento_richiesto CHECK (
    TipoDocumento IN ('Passaporto', 'Carta Identit '))
18  CONSTRAINT volo_internazionale_codicegate_fkey
    FOREIGN KEY (codicegate) REFERENCES gate(codicegate)
19 );
20
21 --TRIGGER SULLA TABELLA
22 CREATE TRIGGER trg_assegna_gate_n
23 BEFORE UPDATE OF StatoVolo ON volo_internazionale
24 FOR EACH ROW
25 EXECUTE FUNCTION assegna_gate_automatico();
26
27 CREATE TRIGGER trg_libera_gate_n
28 AFTER UPDATE OF StatoVolo OR DELETE ON
    volo_internazionale
29 FOR EACH ROW
30 EXECUTE FUNCTION libera_gate_automatico();

```

5.1.6 Tabella Gate

```

1  CREATE TABLE gate (
2  codicegate varchar(5) NOT NULL,
3  terminale integer NOT NULL,
4  statogate varchar(10),
5  CONSTRAINT gate_pkey PRIMARY KEY (codicegate),
6  CONSTRAINT chk_stato_gate CHECK (StatoGate IN ('Aperto',
    'Chiuso', 'Occupato'))
7 );

```

5.2 Funzioni dei Trigger

5.2.1 Tabella Prenotazione

Gestione ripristino posti Questa funzione si occupa di ripristinare il numero di posti disponibili su un volo quando una prenotazione viene cancellata o modificata. Aggiunge 1 al numero di posti disponibili del volo associato in caso di DELETE o UPDATE.

```

1  CREATE OR REPLACE FUNCTION
    gestione_ripristino_posti_unificata()

```

```
2 RETURNS trigger
3 LANGUAGE plpgsql
4 AS $function$
5 BEGIN
6     IF (TG_OP = 'UPDATE') THEN
7         IF (OLD.StatoPrenotazione = 'Confermata' AND NEW.
8             StatoPrenotazione = 'Cancellata') THEN
9             IF NEW.CodiceVoloN IS NOT NULL THEN
10                UPDATE Volo_Nazionale
11                SET NumPostiDisponibili =
12                NumPostiDisponibili + 1
13                WHERE CodiceVoloN = NEW.CodiceVoloN AND
14                NumPostiDisponibili < 150;
15            ELSE
16                UPDATE Volo_Internazionale
17                SET NumPostiDisponibili =
18                NumPostiDisponibili + 1
19                WHERE CodiceVoloIN = NEW.CodiceVoloIN AND
20                NumPostiDisponibili < 150;
21            END IF;
22        END IF;
23        RETURN NEW;
24    ELSIF (TG_OP = 'DELETE') THEN
25        IF (OLD.StatoPrenotazione = 'Confermata') THEN
26            IF OLD.CodiceVoloN IS NOT NULL THEN
27                UPDATE Volo_Nazionale
28                SET NumPostiDisponibili =
29                NumPostiDisponibili + 1
30                WHERE CodiceVoloN = OLD.CodiceVoloN AND
31                NumPostiDisponibili < 150;
32            ELSE
33                UPDATE Volo_Internazionale
34                SET NumPostiDisponibili =
35                NumPostiDisponibili + 1
36                WHERE CodiceVoloIN = OLD.CodiceVoloIN AND
37                NumPostiDisponibili < 150;
38            END IF;
39        END IF;
40        RETURN OLD;
41    END IF;
42 END;
43 $function$;
```

Controllo documenti del passeggero La funzione controlla se il passeggero rispetta i requisiti di documento richiesti dal volo internazionale. In particolare, se il volo richiede il Passaporto, il passeggero deve avere come tipo documento il Passaporto, altrimenti il sistema blocca l'inserimento della prenotazione. Se invece il volo richiede solo la Carta d'Identità, il passeggero può avere sia la Carta d'Identità che il Passaporto, dato che il Passaporto è un documento di livello superiore riconosciuto e valido.

```
1 CREATE OR REPLACE FUNCTION
    controllo_documenti_internazionali()
2 RETURNS trigger
3 LANGUAGE plpgsql
4 AS $function$
5 DECLARE
6     v_doc_richiesto VARCHAR(20);
7     v_doc_passeggero VARCHAR(20);
8 BEGIN
9     -- Scatta solo se é un volo internazionale
10    IF NEW.CodiceVoloIN IS NOT NULL THEN
11
12        -- 1. Recupero il documento richiesto dal volo
13        SELECT TipoDocumento INTO v_doc_richiesto
14        FROM Volo_Internazionale
15        WHERE CodiceVoloIN = NEW.CodiceVoloIN;
16
17        -- 2. Recupero il documento in possesso del
18        passeggero
19        SELECT Documento INTO v_doc_passeggero
20        FROM PASSEGGERO
21        WHERE ID_Passeggero = NEW.ID_Passeggero;
22
23        -- Se il volo richiede Passaporto ma il
24        passeggero ha Carta Identità -> ERRORE
25        IF v_doc_richiesto = 'Passaporto' AND
26        v_doc_passeggero = 'Carta Identità' THEN
27            RAISE EXCEPTION 'Documento non idoneo: il
28            volo % richiede il Passaporto.', NEW.CodiceVoloIN;
29        END IF;
30    END IF;
31
32    RETURN NEW;
33 END;
34 $function$;
```

Controllo completo inserimento di una Prenotazione Questa funzione effettua diversi controlli per rispettare diversi requisiti del sistema. In primis, controlla che il volo abbia la partenza "Napoli", dato che non è possibile prenotare un volo che non parte da questo aeroporto. In caso di volo con partenza da Napoli, il sistema controlla se lo stato del volo non è su "Decollato" o "Cancellato", impedendo al passeggero di effettuare una eventuale prenotazione per un volo già partito o cancellato. Successivamente il sistema controlla la capienza dell'aereo, e se ha almeno un posto disponibile, imposta la prenotazione su "Confermata" e scala di 1 il posto dal volo. In caso contrario, viene visualizzato un messaggio di errore (così come per tutti i casi precedenti) e l'inesimento della prenotazione viene bloccato.

```

1 CREATE OR REPLACE FUNCTION gestione_prenotazione_completa
  ()
2 RETURNS trigger
3 LANGUAGE plpgsql
4 AS $function$
5 DECLARE
6     v_posti_disp INTEGER;
7     v_partenza VARCHAR(20);
8     v_stato VARCHAR(20);
9 BEGIN
10     -- RECUPERO DATI DEL VOLO (Nazionale o Internazionale)
11     IF NEW.CodiceVoloN IS NOT NULL THEN
12         SELECT NumPostiDisponibili, Partenza, StatoVolo
13         INTO v_posti_disp, v_partenza, v_stato
14         FROM Volo_Nazionale WHERE CodiceVoloN = NEW.
CodiceVoloN;
15     ELSE
16         SELECT NumPostiDisponibili, Partenza, StatoVolo
17         INTO v_posti_disp, v_partenza, v_stato
18         FROM Volo_Internazionale WHERE CodiceVoloIN = NEW
.CodiceVoloIN;
19     END IF;
20
21     -- Controllo Aeroporto
22     IF v_partenza != 'Napoli' THEN
23         RAISE EXCEPTION 'Errore: Volo non valido. L''
aeroporto gestito é solo Napoli.';
24     END IF;
25
26     -- Controllo Stato Volo

```

```

27     IF v_stato IN ('Decollato', 'Cancellato') THEN
28         RAISE EXCEPTION 'Errore: Impossibile prenotare un
volo in stato %.', v_stato;
29     END IF;
30
31     -- Controllo Capienza
32     IF v_posti_disp <= 0 THEN
33         RAISE EXCEPTION 'Errore: Volo al completo.';
34     END IF;
35
36     NEW.StatoPrenotazione := 'Confermata';
37
38     -- Scala il posto dal volo corrispondente
39     IF NEW.CodiceVoloN IS NOT NULL THEN
40         UPDATE Volo_Nazionale
41         SET NumPostiDisponibili = NumPostiDisponibili - 1
42         WHERE CodiceVoloN = NEW.CodiceVoloN;
43     ELSE
44         UPDATE Volo_Internazionale
45         SET NumPostiDisponibili = NumPostiDisponibili - 1
46         WHERE CodiceVoloIN = NEW.CodiceVoloIN;
47     END IF;
48
49     RETURN NEW;
50 END;
51 $function$
52 ;

```

5.2.2 Tabella Volo Nazionale/Volo Internazionale

Le funzioni dei trigger delle due tabelle sono equivalenti in quanto si occupano di gestire la logica legata ai gate.

Assegnazione automatica del gate Questa funzione si occupa di aggiornare il gate di un volo, assegnandolo al primo gate disponibile con stato "Aperto", quando il volo viene impostato dallo stato "Programmato" a "in Attesa".

```

1 CREATE OR REPLACE FUNCTION assegna_gate_automatico()
2 RETURNS trigger
3 LANGUAGE plpgsql
4 AS $function$
5 DECLARE

```

```
6      v_id_gate_libero varchar;
7  BEGIN
8
9      IF NEW.StatoVolo = 'in Attesa' AND NEW.CodiceGate IS
      NULL THEN
10
11          SELECT CodiceGate INTO v_id_gate_libero
12          FROM GATE
13          WHERE StatoGate = 'Aperto'
14          ORDER BY CodiceGate ASC
15          LIMIT 1;
16
17      IF v_id_gate_libero IS NOT NULL THEN
18          NEW.CodiceGate := v_id_gate_libero;
19
20          UPDATE GATE
21          SET StatoGate = 'Occupato'
22          WHERE CodiceGate = v_id_gate_libero;
23      ELSE
24
25          RAISE NOTICE 'Attenzione: Nessun Gate
      disponibile per il volo %', COALESCE(NEW.CodiceVoloN,
      NEW.CodiceVoloIN);
26      END IF;
27      END IF;
28
29      RETURN NEW;
30  END;
31  $function$
32  ;
```

Libera automaticamente il gate Questa funzione si occupa il gate assegnato ad un volo, quando lo stato del volo viene impostato su "Decollato" o "Cancellato", oppure viene rimosso dal database. Il volo partito o cancellato mantiene il riferimento al gate, a scopo informativo e di registro, mentre il gate viene liberato e impostato su "Aperto" nuovamente e disponibile per altri voli.

```
1  CREATE OR REPLACE FUNCTION libera_gate_automatico()
2  RETURNS trigger
3  LANGUAGE plpgsql
4  AS $function$
5  BEGIN
```

```
6      -- CASO 1: DELETE (Il volo viene rimosso fisicamente)
7      IF (TG_OP = 'DELETE') THEN
8          IF OLD.CodiceGate IS NOT NULL THEN
9              UPDATE GATE SET StatoGate = 'Aperto' WHERE
10             CodiceGate = OLD.CodiceGate;
11             END IF;
12             RETURN OLD;
13
14     -- CASO 2: UPDATE (Il volo cambia stato in Decollato
15     o Cancellato)
16     ELSIF (TG_OP = 'UPDATE') THEN
17         IF NEW.StatoVolo IN ('Decollato', 'Cancellato')
18         AND OLD.CodiceGate IS NOT NULL THEN
19             UPDATE GATE SET StatoGate = 'Aperto' WHERE
20             CodiceGate = OLD.CodiceGate;
21             END IF;
22             RETURN NEW;
23             END IF;
24
25     RETURN NULL;
26 END;
27 $function$
28 ;
```